

# Potato Disease Classification

**Sawon Paul**

Department of Computer Science and Engineering  
BRAC University

**Supervisor:** Md. Saiful Islam

June 2025

# 1 Introduction

Potato is a staple crop consumed worldwide, and its health is crucial for food security. However, potato plants are susceptible to several diseases, the most common being Early Blight and Late Blight.

Early Blight is caused by a fungus called *Alternaria solani*, while Late Blight is caused by the oomycete *Phytophthora infestans*. These diseases can severely reduce crop yield and quality, making early detection essential for effective disease management.

This project focuses on building a Convolutional Neural Network (CNN) model to automatically classify potato leaf images into three categories: Early Blight, Late Blight, and Healthy. Using deep learning techniques enables faster, scalable, and more accurate detection compared to manual inspection.

All the code components used in this project—from image preprocessing, dataset handling, model architecture design, training, evaluation, and deployment—are explained in detail throughout this document.

Key libraries such as TensorFlow, Keras, and Matplotlib, along with specific methods, parameters, and model layers, have been discussed to provide a comprehensive understanding of the pipeline used for potato disease classification.

- **TensorFlow (tf):** It is an open-source machine learning framework by Google for building and training neural networks, using dataflow graphs where tensors (multi-dimensional arrays) flow through operations. Its scalability and flexibility make it ideal for image classification tasks like potato disease detection. It supports efficient data handling and deployment.
- **tensorflow.keras:** tensorflow.keras is the high-level API within TensorFlow, a popular open-source machine learning framework developed by Google. It is an implementation of the Keras API, integrated directly into TensorFlow (since TensorFlow 2.0) to simplify the process of building, training, and deploying machine learning models, particularly deep neural networks. Keras provides a user-friendly interface for defining neural network architectures, handling data preprocessing, and managing training workflows, while leveraging TensorFlow's backend for efficient computation and scalability.
- **tensorflow.keras.models:** A Keras module providing classes like `Sequential` for building neural network models. Simplifies CNN construction for classifying potato leaf images by stacking layers.
- **tensorflow.keras.layers:** A Keras module with layer types (e.g., `Conv2D`, `Dense`) for neural networks. Enables building a CNN with convolutional and pooling layers suited for image processing.
- **matplotlib.pyplot:** A plotting library for visualizing data, aliased as `plt`. Visualizes test images and predictions to evaluate model performance.
- **IMAGE\_SIZE = 256:** Resizes images to 256x256 pixels. Standardizes input for the CNN, balancing detail and computation.

- **BATCH\_SIZE = 32**: Processes 32 images per training iteration. Optimizes memory usage and training stability.
- **CHANNELS = 3**: Specifies RGB images (3 color channels). Matches the RGB input format for the model.
- **EPOCHS = 50**: Sets 50 training iterations over the dataset. Allows sufficient learning iterations while risking overfitting if too high.
- **tf.keras.preprocessing**: It is a module within TensorFlow's Keras API that provides utilities for preprocessing data, particularly for preparing datasets for machine learning tasks. It includes tools for handling images, text, and sequences, making it easier to transform raw data into a format suitable for training neural networks. In the context of the training.ipynb notebook, tf.keras.preprocessing is used to load and preprocess image data from the PlantVillage dataset for a convolutional neural network (CNN) to classify potato leaf images into three categories: Early Blight, Late Blight, and Healthy.
- **image\_dataset\_from\_directory**: A TensorFlow utility that loads images from a directory, where subdirectories represent classes, into a `tf.data.Dataset` object. Efficiently loads and preprocesses the PlantVillage dataset (2152 images, 3 classes), handling resizing, batching, and shuffling for training.
- **shuffle = True**: Randomizes image order to improve model generalization.
- **dataset.class\_names**: A property returning class names from dataset subdirectories. Maps numerical labels to class names (e.g., `Potato___Early_blight`) for interpreting predictions.
- **len(dataset)**: It returns the number of batches (2152 images / 32 = 68 batches). Verifies dataset size for splitting into training, validation, and test sets.
- **plt.figure(a,b)**: It creates an  $a \times b$  inch figure.
- **test\_ds.take(1)**: Retrieves one batch from the test dataset.
- **plt.subplot**: Creates a 3x3 grid for plotting.
- **plt.imshow**: Displays an image after converting to NumPy and uint8.
- **predict**: A custom function returning predicted class and confidence.
- **class\_name[labels[i]]**: Maps numerical labels to class names.
- **plt.title**: Shows actual and predicted labels with confidence.
- **plt.axis("off")**: Hides axes for clarity.

- **plt.legend**: Displays a legend on a Matplotlib plot, describing each plotted data series. It uses labels specified in `plt.plot(..., label="...")`.
- **get\_dataset\_partitions\_tf**: Purpose of this function is to splits a TensorFlow dataset (ds) into three parts: training (train\_ds), validation (val\_ds), and test (test\_ds) datasets based on specified proportions.
- **cache()**: Caches the dataset in memory after the first epoch (or to disk if a filename is passed). It avoids reloading images from disk in every epoch, speeding up training significantly. Useful for datasets that don't change and are small enough to fit in memory.
- **shuffle(1000)**: It randomly shuffles the dataset and prevents the model from learning the order of the data and helps generalize better. 1000 is the buffer size which is a temporary list from which elements are randomly selected. Larger values mean better shuffling but use more memory.
- **prefetch(buffer\_size=tf.data.AUTOTUNE)**: It allows the data pipeline to fetch the next batch while the current one is being processed by the model. It overlaps data preparation and model execution, improving GPU/CPU utilization. TensorFlow automatically determines the best buffer size to maximize performance.
- **Sequential()**: It allows to stack preprocessing layers in a pipeline.
- **tf.keras.layers.Resizing(IMAGE\_SIZE, IMAGE\_SIZE)**: Resizes all input images to a fixed size — (IMAGE\_SIZE, IMAGE\_SIZE) (e.g.,  $256 \times 256$ ). Deep learning models require all inputs to have the same dimensions. This layer helps standardize images across datasets with varying sizes.
- **tf.keras.layers.Rescaling(1.0/255)**: Scales pixel values from the range [0, 255] (standard RGB) to [0.0, 1.0]. Neural networks perform better when input values are small and normalized.
- **tf.keras.layers.RandomFlip("horizontal\_and\_vertical")**: Randomly flips the image horizontally, vertically, or both. This helps the model generalize better by making it invariant to image orientation.
- **tf.keras.layers.RandomRotation(0.2)**: Randomly rotates the image by a factor of  $0.2 \times 2\pi$  (i.e., up to approximately  $\pm 72^\circ$ ). This improves model robustness to slight rotations that are common in real-world image capture.
- **Conv2D()**: It's the most fundamental layer in CNNs stands for 2D Convolutional Layer. It is used to extracts features (like edges, corners, textures) from input images using filters (kernels) and each filter slides across the image to create a feature map (activation map).
- **filters=64**: Number of feature detectors.

- **kernel\_size=(3,3)**: Size of the sliding window.
- **activation='relu'**: Adds non-linearity to the model.
- **MaxPooling2D()**: Reduces the spatial dimensions (height  $\times$  width) of the feature maps. It decreases computational cost, helps prevent overfitting, and retains the most important features.
- **Flatten()**: Converts multi-dimensional tensor (e.g.,  $64 \times 64 \times 3$ ) into a 1D vector. It is needed before feeding into Dense (fully connected) layers. It is also used before classification layers and it acts like a "bridge" from CNN to ANN layers.
- **Dense()**: It is a regular neural network layer where every input neuron is connected to every output neuron. It is used to perform classification or regression and learns complex combinations of features.
- **Relu**: ReLU stands for Rectified Linear Unit, it keeps all positive values and zeros out negative values. It is used in almost all hidden layers in CNN's, RNNs, etc and also used in deep autoencoders and generative models.
- **Softmax**: Softmax turns raw output values (logits) into probabilities that sum up to 1. It is used only in the output layer where there are multiple classes and probabilities of each class are paired with categorical cross-entropy loss.
- **model.build()**: It manually builds the model's internal structure before training or using `.summary()`.
- **model.summary()**: It is used in TensorFlow/Keras to print a summary of the model architecture, showing: each layer type, output shape of each layer, number of parameters (weights + biases), total trainable and non-trainable parameters.
- **model.compile()**: It is a function in TensorFlow/Keras used to configure the model for training. It defines: the optimizer — how the model updates weights, the loss function — what the model tries to minimize, the evaluation metrics — what to monitor during training/testing.
- **optimizer='adam'**: Adam is a popular and efficient optimization algorithm that combines momentum and RMSProp. It adjusts the learning rate adaptively for each parameter and is very commonly used in CNNs and other deep learning tasks.
- **loss=tf.keras.losses.SparseCategoricalCrossentropy(from\_logits=False)**: This is used for multi-class classification where labels are integers (e.g., 0, 1, 2), not one-hot vectors, and the output layer uses **softmax** as the activation function. If **from\_logits=True**, then the loss expects raw scores (logits) instead of probabilities from **softmax**.

- **metrics=['accuracy']**: This tells Keras to display accuracy during training and evaluation. It compares the predicted class (i.e., the class with the highest **softmax** score) with the true class label.
- **model.fit()**: This is main function to train a neural network.
- **epochs**: The number of times the model will see the entire training dataset during training.
- **batch\_size**: The number of samples processed before the model's weights are updated.
- **verbose=1**: Controls how much output you see while training. 0 for silent, 1 for progress bar (default), 2 for one line per epoch.
- **validation\_data**: This is used to evaluate the model after each epoch on unseen validation data.
- **model.evaluate()**: It runs inference on the test dataset, calculates the loss and metrics (e.g., accuracy) and returns them as a list (e.g., [loss, accuracy]).
- **.params**: When a model is train using Keras then it returns a object and one of its properties is params. This dictionary tells what training configuration was used — it helps inspect or debug the training process.
- **.astype('uint8')**: Converts a NumPy array to unsigned 8-bit integers (0 to 255), the standard format for image pixel values.
- **images\_batch[0].numpy().astype('uint8')**: Takes the first image in a batch, converts it from a Tensor to a NumPy array, and changes the data type to uint8 for proper image display.
- **labels\_batch[0].numpy()**: Extracts the first label from a batch of labels (**labels\_batch[0]**) and converts it from a Tensor to a NumPy array.
- **.argmax()**: It is used to find the index of the maximum value in an array. It's commonly used in NumPy, PyTorch, TensorFlow, and similar libraries.
- **class\_name[np.argmax(batch\_prediction[0])]**:
  - **batch\_prediction[0]**: First prediction (class probabilities) from the model.
  - **np.argmax(...)**: Index of the highest probability.
  - **class\_name[...]**: Retrieves the corresponding class label.
- **model.predict()**  
Generates predictions from the model for the given input data. Outputs could be class probabilities or regression results.

- `round(100*(np.max(predictions[0])), 2)`
  - `np.max(predictions[0])`: Highest probability (confidence) from the first prediction.
  - Multiply by 100 to convert to a percentage.
  - Round to two decimal places.
- `tf.expand_dims(img_array, 0)`: Adds a new dimension at index 0 (batch dimension), converting an image of shape  $(H, W, C)$  into  $(1, H, W, C)$ , as models require a batch dimension.
- `tf.keras.preprocessing.image.img_to_array(images[i].numpy())`: Converts `images[i]` from a Tensor to a NumPy array, and then into a Keras-compatible array format for model input.
- `model.export()`: It saves the model in SavedModel format for deployment. enables reuse and deployment (e.g., TensorFlow Serving, TensorFlow Lite) of the trained model for potato disease classification. Confirms model saved to `../models/1`.
- `os.listdir()`: It is a Python function listing directory contents. Verifies saved model files and directory structure for future use.
- `model.save()`: It saves the model in `.keras` or `.h5` format. Intended to save another model version, but fails due to a missing file extension (requires `.keras` or `.h5`).